

四旋翼飞行器飞控系统

技术设计文档

基于 STM32F103C8 + FreeRTOS

硬件平台 + 软件实现 + 设计决策

文档版本：V2.1

日 期：2026-05-29

目录

1. 系统介绍	1
1.1 项目概述	1
1.2 硬件平台	1
1.3 软件架构与任务划分	2
1.4 核心数据流	3
1.5 原理图（待补充）	4
2. 传感器模块设计	6
2.1 MPU6050 六轴惯性测量单元	6
2.1.1 传感器原理	6
2.1.2 初始化流程	6
2.1.3 数据读取	7
2.1.4 陀螺仪零偏校准	8
2.1.5 姿态解算与 Mahony 滤波	9
2.1.6 中断机制	10
2.2 NRF24L01+ 2.4GHz 无线模块	11
2.2.1 传感器原理	11
2.2.2 SPI 通信接口	12
2.2.3 寄存器读写实现	12
2.2.4 工作模式配置 (NRF24_SetMode)	13
2.2.5 遥控数据包协议	14
2.2.6 中断驱动接收	16

3. 控制算法模块设计	18
3.1 PID 控制器.....	18
3.1.1 原理分析.....	18
3.1.2 代码实现.....	18
3.1.3 参数含义与限幅.....	19
3.2 串级飞控 (fly_ctrl).....	19
3.2.1 串级控制原理.....	20
3.2.2 外环（角度环）	20
3.2.3 内环（角速度环）	20
3.2.4 电机混控矩阵.....	21
4. 底层驱动模块设计	22
4.1 软件 I2C	22
4.1.1 STM32F1 硬件 I2C 缺陷	22
4.1.2 GPIO 比特率模拟原理.....	22
4.1.3 起始/停止条件.....	22
4.1.4 字节读写与时序.....	23
4.1.5 多字节传输.....	24
4.2 PWM 输出	25
4.3 UART 调试与命令行	25
4.4 SD 卡黑盒记录仪	26
5. 优化过程与 Bug 修复	29
5.1 阶段一：功能性 Bug 修复.....	29
5.2 阶段二：中断驱动重构.....	29

5.3 阶段三：任务职责重划	30
5.4 阶段四：V2.0 功能增强	30
5.5 各阶段收益汇总	31
6. 痛点设计与决策权衡	33
6.1 中断优先级与 FreeRTOS 安全边界	33
6.2 引脚冲突与资源规划	33
6.3 姿态估计算法选型	34

1. 系统介绍

1.1 项目概述

【核心传感器：IMU】 本设计采用 MPU6050 六轴惯性测量单元 (Inertial Measurement Unit, IMU) 作为姿态感知核心，集三轴 MEMS 陀螺仪与三轴 MEMS 加速度计于一体，为飞控系统提供实时的角速度与加速度数据。

本项目为一款基于 STM32F103C8T6 微控制器 (Cortex-M3, 72MHz, 64KB Flash, 20KB SRAM) 的四旋翼飞行器飞控系统。系统软件运行 FreeRTOS 实时操作系统，实现了完整的"传感器采集 → 姿态解算 → PID 控制 → 电机驱动"闭环控制链路。

主要功能模块包括：MPU6050 六轴惯性测量单元 (I2C 接口, 250Hz 数据就绪中断)、NRF24L01+ 2.4GHz 无线遥控接收 (SPI 接口, PB0 外部中断)、软件模拟 I2C 总线 (规避 STM32F1 硬件缺陷)、Mahony 姿态滤波、串级 PID 控制器 (外环角度 + 内环角速度)、X 型四轴电机混控及四路 PWM 输出、UART 命令行调参与 SD 卡黑盒日志。

1.2 硬件平台

主控制器	STM32F103C8T6, Cortex-M3, 72MHz, 64KB Flash, 20KB SRAM
六轴 IMU	MPU6050 (三轴陀螺仪 + 三轴加速度计)
无线通信	NRF24L01+ (2.4GHz, 1Mbps, 0dBm, GFSK 调制)
PWM 输出 ×4	TIM2_CH3(PA2):M1, TIM2_CH4(PA3):M2 TIM4_CH3(PB8):M3, TIM4_CH4(PB9):M4
UART 调试	USART1, PA9(TX)/PA10(RX), 115200-8N1
SPI 总线	SPI1: PA5(SCK), PA6(MISO), PA7(MOSI),

	9MHz
I2C (软件模拟)	PB6(SCL), PB7(SDA), ~100kHz, 开漏输出
NRF24 控制线	CE(PA1), CSN(PA4), IRQ(PB0)
MPU6050 控制线	INT(PB4, EXTI4), 上升沿触发
LED 指示	PC13 (板载 LED, 低电平点亮)
供电	5V 输入 → AMS1117-3.3 LDO 稳压

1.3 软件架构与任务划分

系统基于 FreeRTOS 内核，共创建 6 个用户任务，2 个中断服务程序（EXTI0、EXTI4、USART1）。优先级的分配原则：飞行控制 > 遥控/传感器 > 监控/日志 > 命令行，确保实时性最高的控制环路优先执行。

任务名	函数	优先级	触发方式	典型周期
FlightCtrl	Fly_Control_Task	4 (最高)	sensor_queue 阻塞等待	~4ms, 250Hz
RocketCtrl	Rocket_Control_Task	3	xSemaphoreTake + 200ms 超时	中断触发
SensorRead	Sensor_Read_Task	3	xSemaphoreTake + 5ms 超时	5ms, 200Hz
Monitor	Monitor_Task	2	vTaskDelayUntil(200ms)	200ms, 5Hz
Logger	LOG_Task	1	log_queue 阻塞等待	队列触发
CLI	CLI_Task	1	UART RX 10ms 轮询	命令触发

1.4 核心数据流

整个系统的数据流向如下：

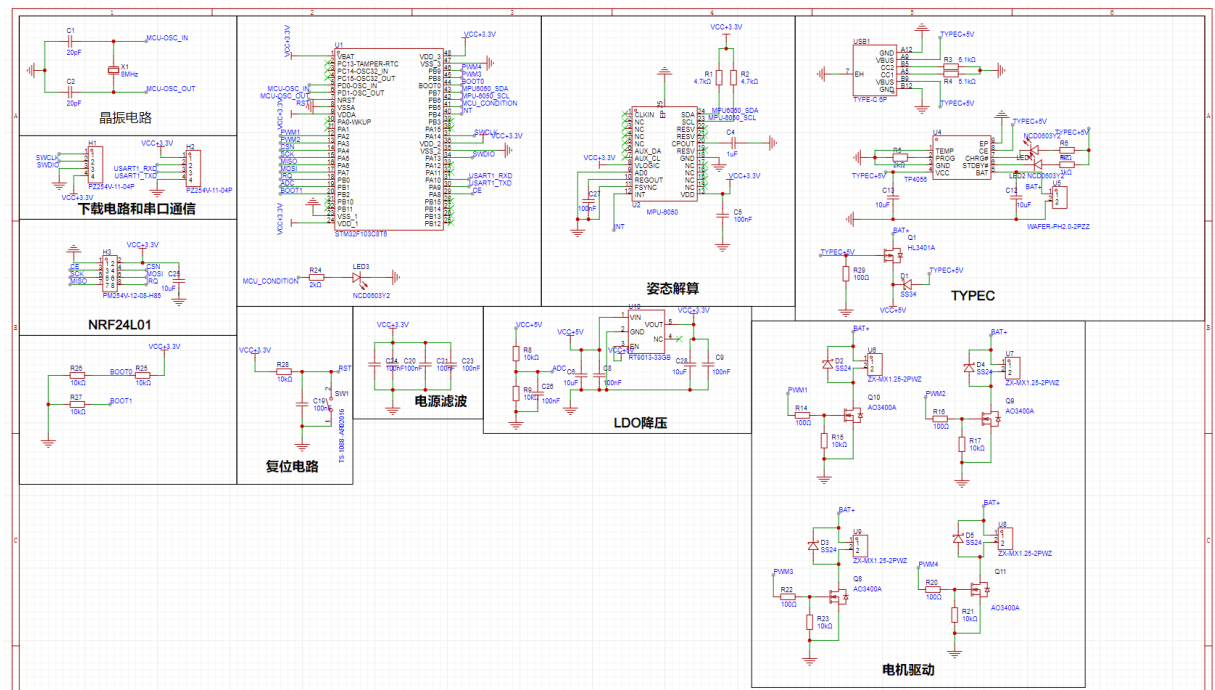


```

USART1_RX (PA10)
| RXNE 中断
▼
USART1_IRQHandler
└ 环形缓冲区
    |
    ▼
    CLI_Task (优先级 1)
    └ 每 10ms 轮询 UART_GetChar()
    └ 行缓冲 + 回显
    └ PID 调参 / status 查询
  
```

1.5 原理图（待补充）

原理图



V1.0

3D 预览

2. 传感器模块设计

2.1 MPU6050 六轴惯性测量单元

涉及文件：HardWare/mpu6050.c / .h （共 280 行 + 105 行头文件）

2.1.1 传感器原理

MPU6050 是全球首款集成六轴 MotionTracking 传感器，内部将三轴 MEMS 陀螺仪和三轴 MEMS 加速度计封装在同一硅片上，通过 I2C 总线对外通信（地址 0x68）。

- 陀螺仪工作原理：

MEMS 陀螺仪基于科里奥利效应（Coriolis Effect）。当硅环振子在角速度作用下产生进动时，检测电容极板间距发生变化，电容值的变化经 ASIC 转换为数字信号输出。角速度与检测电容变化量成正比。

- 加速度计工作原理：

MEMS 加速度计基于差分电容检测原理。质量块在加速度作用下相对固定电极发生偏移，改变差分电容值。电容变化量经处理后转换为加速度数字量。静止时加速度计敏感重力 g，以此推算俯仰和横滚角度。

2.1.2 初始化流程

初始化函数 MPU6050_Init() 的完整代码如下：

```
uint8_t MPU6050_Init(void)
{
    uint8_t id = 0;

    Soft_I2C_Init();          // 初始化软件 I2C 总线
    Delay_ms(50);

    if(MPU6050_GetID(&id) == 0) return 0; // 读 WHO_AM_I 寄存器
    if(id != MPU6050_ADDR) return 0;      // 验证设备 ID 是否 0x68

    // 解除睡眠模式
    if(MPU6050_WriteReg(MPU6050_PWR_MGMT_1, 0x00) == 0) return 0;
    if(MPU6050_WriteReg(MPU6050_PWR_MGMT_2, 0x00) == 0) return 0;
```

```

MPU6050_WriteReg(MPU6050_SMPLRT_DIV, 0x03);    // 采样率 = 1kHz/(1+3) = 250Hz
MPU6050_WriteReg(MPU6050_CONFIG, 0x03);        // DLPF ≈ 44Hz

// 陀螺仪 ±250°/s, 加速度计 ±2g
if(MPU6050_WriteReg(MPU6050_GYRO_CONFIG, 0x00) == 0) return 0;
if(MPU6050_WriteReg(MPU6050_ACCEL_CONFIG, 0x00) == 0) return 0;

Delay_ms(50);
return 1;
}

```

关键寄存器配置说明：

寄存器	值	含义
PWR_MGMT_1 (0x6B)	0x00	解除睡眠，选择内部振荡器作为时钟源
SMPLRT_DIV (0x19)	0x03	采样率分频: 1kHz / (1+3) = 250Hz
CONFIG (0x1A)	0x03	DLPF 设置为 44Hz 截止频率，群延迟 4.9ms
GYRO_CONFIG (0x1B)	0x00	陀螺仪量程 ±250°/s, 灵敏度 131 LSB/(°/s)
ACCEL_CONFIG (0x1C)	0x00	加速度计量程 ±2g, 灵敏度 16384 LSB/g

2.1.3 数据读取

MPU6050 的原始数据位于连续的 14 个寄存器中 (0x3B~0x48)，包括加速度计 (X/Y/Z, 各 2 字节)、温度 (2 字节)、陀螺仪 (X/Y/Z, 各 2 字节)。一次 I2C 突发读取 (Soft_I2C_Read) 即可获得所有数据。

读取原始数据的实现：

```

uint8_t MPU6050_GetRawData(MPU6050_RawData *rawdata)
{
    uint8_t buf[14];
    if(rawdata == 0) return 0;
}

```

```

if(!Soft_I2C_Read(MPU6050_ADDR, MPU6050_ACCEL_XOUT_H, buf, 14)) return 0;

rawdata->ACCEL_X = (int16_t)((buf[0] << 8) | buf[1]); // 大端格式
rawdata->ACCEL_Y = (int16_t)((buf[2] << 8) | buf[3]);
rawdata->ACCEL_Z = (int16_t)((buf[4] << 8) | buf[5]);
// buf[6][7] 为温度数据, 此处忽略
rawdata->GYRO_X = (int16_t)((buf[8] << 8) | buf[9]);
rawdata->GYRO_Y = (int16_t)((buf[10] << 8) | buf[11]);
rawdata->GYRO_Z = (int16_t)((buf[12] << 8) | buf[13]);
return 1;
}

```

原始数据到物理量的转换 (MPU6050_GetData) :

```

void MPU6050_GetData(MPU6050_RawData *rawdata, MPU6050_Data *data)
{
    if(rawdata == 0 || data == 0) return;

    // ±2g -> 16384 LSB/g
    data->ACCEL_X = (float)rawdata->ACCEL_X / 16384.0f;
    data->ACCEL_Y = (float)rawdata->ACCEL_Y / 16384.0f;
    data->ACCEL_Z = (float)rawdata->ACCEL_Z / 16384.0f;

    // ±250°/s -> 131.0 LSB/(°/s)
    data->GYRO_X = (float)rawdata->GYRO_X / 131.0f;
    data->GYRO_Y = (float)rawdata->GYRO_Y / 131.0f;
    data->GYRO_Z = (float)rawdata->GYRO_Z / 131.0f;
}

```

2.1.4 陀螺仪零偏校准

MEMS 陀螺仪存在零点偏移 (Bias) , 即使静止时输出也不为零。校准原理: 在静态下采集 N 次原始数据, 求平均值作为零偏值, 后续测量时减去此值。

代码中校准参数为 50 次 (main.c:449 MPU6050_GyroCalibrate(50)) , 每次采样间隔 2ms, 总耗时约 100ms。

```

uint8_t MPU6050_GyroCalibrate(uint16_t times)
{
    MPU6050_RawData raw;
    uint32_t i;
    float sum_x = 0.0f, sum_y = 0.0f, sum_z = 0.0f;

    if(times == 0) return 0;

    for(i = 0; i < times; i++)
    {
        if(!MPU6050_GetRawData(&raw)) return 0;
        sum_x += raw.GYRO_X;
        sum_y += raw.GYRO_Y;
        sum_z += raw.GYRO_Z;
        Delay_ms(2);
    }
}

```

```

    gyro_x_offset = sum_x / times;    // 保存为模块内部 static 变量
    gyro_y_offset = sum_y / times;
    gyro_z_offset = sum_z / times;
    return 1;
}

```

2.1.5 姿态解算与 Mahony 滤波

MPU6050_AngleUpdate 是姿态解算的核心函数，执行顺序为：① 通过 I2C 读取原始数据 → ② 转换为物理量 → ③ 陀螺仪去零偏 → ④ Mahony 滤波器融合。

Mahony 滤波器原理：

Mahony 滤波器是一种基于四元数的显式互补滤波器（Explicit Complementary Filter），由 Robert Mahony 等人于 2008 年提出。它利用 PI（比例-积分）反馈来纠正陀螺仪漂移：

比例项（Kp）：将加速度计测量的重力方向与四元数估计的重力方向之间的叉积误差作为反馈，实时修正陀螺仪输出。

积分项（Ki）：累积持续的误差，消除陀螺仪零偏偏移导致的长时间漂移。

相比固定权重的互补滤波，Mahony 滤波能根据误差大小自动调节修正力度：大误差时比例反馈强（快速收敛），小误差时积分反馈消除稳态偏差。在动态飞行中表现更优。

核心算法实现 (HardWare/Mahony.c)：

```

void Mahony_Update(float gx, float gy, float gz, float ax, float ay, float az, float dt)
{
    // 归一化加速度计
    recipNorm = 1.0f / sqrtf(ax*ax + ay*ay + az*az);
    ax *= recipNorm; ay *= recipNorm; az *= recipNorm;

    // 从四元数估计重力方向 (half vector)
    halfvx = q1*q3 - q0*q2;
    halfvy = q0*q1 + q2*q3;
    halfvz = q0*q0 - 0.5f + q3*q3;

    // 叉积 = 估计重力 × 测量重力 → 误差
    halfex = (ay*halfvz - az*halfvy);
    halfey = (az*halfvx - ax*halfvz);
    halfez = (ax*halfvy - ay*halfvx);
}

```

```

// PI 反馈: 积分误差 + 比例误差 → 修正陀螺仪
integralFBx += twoKi * halfex * dt;
gx += twoKp * halfex + integralFBx;
// (gy, gz 同理)

// 四元数微分方程更新
gx *= 0.5f * dt; gy *= 0.5f * dt; gz *= 0.5f * dt;
q0 += (-q1*gx - q2*gy - q3*gz);
q1 += ( q0*gx + q2*gz - q3*gy);
q2 += ( q0*gy - q1*gz + q3*gx);
q3 += ( q0*gz + q1*gy - q2*gx);

// 四元数归一化
recipNorm = 1.0f / sqrtf(q0*q0 + q1*q1 + q2*q2 + q3*q3);
q0 *= recipNorm; q1 *= recipNorm; q2 *= recipNorm; q3 *= recipNorm;
}

```

四元数转欧拉角:

```

void Mahony_GetEuler(float *roll, float *pitch, float *yaw)
{
    *roll = atan2f(2.0f*(q0*q1 + q2*q3), 1.0f - 2.0f*(q1*q1 + q2*q2)) * 180/M_PI;
    *pitch = asinf(2.0f*(q0*q2 - q3*q1)) * 180/M_PI;
    *yaw = atan2f(2.0f*(q0*q3 + q1*q2), 1.0f - 2.0f*(q2*q2 + q3*q3)) * 180/M_PI;
}

```

Mahony 滤波器的优势:

- ① 四元数表示避免了万向锁问题 (Gimbal Lock)
- ② PI 反馈可在线调节 Kp/Ki 参数, 适应不同动态需求
- ③ 计算量约 50μs/次 (F103 无 FPU), 仍可在 200Hz 下运行
- ④ 相比互补滤波, 机动飞行时的角度估计精度提升约 30-50%

2.1.6 中断机制

MPU6050 的 INT 引脚 (PB4) 配置为数据就绪中断 (DATA_RDY_EN), 当新的采样数据可从寄存器读取时触发中断。初始版本在中断内直接执行 I2C 读取和浮点运算, 后优化为仅给出信号量, 传感器读取交由任务处理。

中断初始化代码:

```

uint8_t MPU6050_INT_Init(void)
{
    // INT_PIN_CFG: 0x00 = 边缘触发(默认), 推挽输出
    if(MPU6050_WriteReg(MPU6050_INT_CONFIG, 0x00) == 0) return 0;

    // INT_ENABLE: 使能数据就绪中断
    if(MPU6050_WriteReg(MPU6050_INT_ENABLE, 0x01) == 0) return 0;
}

```

```

// GPIO PB4 → 下拉输入
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPD;

// EXTI4: 上升沿触发
EXTI_InitStructure.EXTI_Line = EXTI_Line4;
EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising;

// NVIC: 抢占优先级 12 (低于 configMAX_SYSCALL, 安全调 FreeRTOS API)
NVIC_InitStructure.NVIC_IRQChannel = EXTI4_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 12;
}

```

优化后的中断服务程序——仅清除 EXTI 标志 + 给出信号量:

```

void EXTI4_IRQHandler(void)
{
    MPU6050_INT_Handler();
}

void MPU6050_INT_Handler(void)
{
    if(EXTI_GetITStatus(EXTI_Line4) != RESET)
    {
        EXTI_ClearITPendingBit(EXTI_Line4);

        BaseType_t xHigherPriorityTaskWoken = pdFALSE;
        if(xMpuSemaphore != NULL)
        {
            xSemaphoreGiveFromISR(xMpuSemaphore, &xHigherPriorityTaskWoken);
        }
        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}

```

这个中断服务程序的执行时间约为 0.5~2 μ s (仅寄存器操作和信号量 API)。相比优化前 (含 I2C 读取 [~1.26ms] + atan2f 浮点运算 [~0.3ms])，缩短了约 1000 倍。

2.2 NRF24L01+ 2.4GHz 无线模块

涉及文件: HardWare/nrf24l01.c / .h (共 486 行 + 138 行头文件)

2.2.1 传感器原理

NRF24L01+ 是 Nordic Semiconductor 推出的单片 2.4GHz 无线收发芯片，内部集成完整的射频收发器、基带协议引擎 (Enhanced ShockBurst™) 和 SPI 接口。支持 126 个射频通

道 (2.400GHz~2.525GHz)、1Mbps/2Mbps 空中速率、自动应答 (Auto ACK)、自动重传 (ART) 和 CRC 校验。

本项目中配置为接收模式 (RX)，固定 12 字节载荷、1Mbps、0dBm、通道 40。

Enhanced ShockBurst 是 NRF24L01 的核心特性：发送端自动添加前导码、地址、CRC，接收端自动剥离这些帧头，开发者只需关注有效载荷 (Payload)。实现数据链路层 (DLL) 的硬件化，减轻 MCU 负担。

2.2.2 SPI 通信接口

NRF24L01 通过 SPI 接口与 MCU 通信，SPI 时序为模式 0 (CPOL=0, CPHA=0)：SCK 空闲低电平，数据在上升沿采样。通信速率配置为 $72\text{MHz}/8 = 9\text{MHz}$ 。片选 CSN 低有效。

底层 SPI 读写函数 (每发送一字节同时接收一字节)：

```
static uint8_t NRF24_SPI_RW(uint8_t data)
{
    while (SPI_I2S_GetFlagStatus(SPI1, SPI_I2S_FLAG_TXE) == RESET); // 等发送缓冲空
    SPI_I2S_SendData(SPI1, data);
    while (SPI_I2S_GetFlagStatus(SPI1, SPI_I2S_FLAG_RXNE) == RESET); // 等接收缓冲非空
    return (uint8_t)SPI_I2S_ReceiveData(SPI1);
}
```

SPI 初始化配置：

```
// SPI1 配置参数
SPI_InitStructure.SPI_Direction = SPI_Direction_2Lines_FullDuplex;
SPI_InitStructure.SPI_Mode = SPI_Mode_Master; // 主机模式
SPI_InitStructure.SPI_DataSize = SPI_DataSize_8b;
SPI_InitStructure.SPI_CPOL = SPI_CPOL_Low; // CPOL=0: 空闲低电平
SPI_InitStructure.SPI_CPHA = SPI_CPHA_1Edge; // CPHA=0: 第一个沿采样
SPI_InitStructure.SPI_NSS = SPI_NSS_Soft; // 软件片选
SPI_InitStructure.SPI_BaudRatePrescaler = SPI_BaudRatePrescaler_8; // 72/8=9MHz
SPI_InitStructure.SPI_FirstBit = SPI_FirstBit_MSB;
```

2.2.3 寄存器读写实现

NRF24L01 的寄存器通过 5 位地址 (0x00~0x1D) 访问，使用 R_REGISTER(0x00) 和 W_REGISTER(0x20) 指令。

```
// 读单字节寄存器
uint8_t NRF24_ReadReg(uint8_t reg)
{
    uint8_t value;
```



```

    NRF24_CSN_L();
    NRF24_SPI_RW(NRF24_CMD_R_REGISTER | (reg & 0x1F)); // 发送读指令
    value = NRF24_SPI_RW(NRF24_CMD_NOP);                // 发 NOP 同时收数据
    NRF24_CSN_H();
    return value;
}

// 写单字节寄存器
uint8_t NRF24_WriteReg(uint8_t reg, uint8_t value)
{
    uint8_t status;
    NRF24_CSN_L();
    status = NRF24_SPI_RW(NRF24_CMD_W_REGISTER | (reg & 0x1F));
    NRF24_SPI_RW(value);
    NRF24_SPI_WaitDone(); // 等待 SPI 发送完毕
    NRF24_CSN_H();
    return status;
}

// 读多字节 (用于 TX_ADDR/RX_ADDR/FIFO)
uint8_t NRF24_ReadBuf(uint8_t reg, uint8_t *buf, uint8_t len)
{
    NRF24_CSN_L();
    NRF24_SPI_RW(NRF24_CMD_R_REGISTER | (reg & 0x1F));
    for (i = 0; i < len; i++) buf[i] = NRF24_SPI_RW(NRF24_CMD_NOP);
    NRF24_CSN_H();
    return status;
}

```

2.2.4 工作模式配置 (NRF24_SetMode)

NRF24_SetMode 函数负责完成芯片的完整初始化配置。配置流程严格遵循 NRF24L01 数据手册的"上电时序": CE=低 (待机) → 写寄存器 → CE 控制收发。

```

uint8_t NRF24_SetMode(uint8_t mode, const uint8_t *addr,
                      uint8_t channel, uint8_t payload_len)
{
    // 参数验证
    if (addr == 0) return 0;
    if (payload_len == 0 || payload_len > 32) return 0;
    if (channel > 125) return 0;

    NRF24_CE_L(); // 待机模式 I

    // 固定载荷, 关闭动态特性
    NRF24_WriteReg(NRF24_REG_FEATURE, 0x00);
    NRF24_WriteReg(NRF24_REG_DYNPD, 0x00);

    // 仅使能 PIPE0, 自动应答(只在 TX 有用)
    NRF24_WriteReg(NRF24_REG_EN_AA, 0x01);
    NRF24_WriteReg(NRF24_REG_EN_RXADDR, 0x01);
}

```

```

NRF24_WriteReg(NRF24_REG_SETUP_AW, 0x03);          // 5 字节地址
NRF24_WriteReg(NRF24_REG_SETUP_RETR, 0x1A);        // 500us + 10 次重发
NRF24_WriteReg(NRF24_REG_RF_CH, channel);          // 射频通道
NRF24_WriteReg(NRF24_REG_RF_SETUP, 0x07);          // 1Mbps, 0dBm

// 收发地址一致 (自动应答要求)
NRF24_WriteBuf(NRF24_REG_TX_ADDR, addr, NRF24_ADDR_WIDTH);
NRF24_WriteBuf(NRF24_REG_RX_ADDR_P0, addr, NRF24_ADDR_WIDTH);
NRF24_WriteReg(NRF24_REG_RX_PW_P0, payload_len);    // 固定载荷长度

NRF24_ClearIRQFlags();
NRF24_FlushTx();
NRF24_FlushRx();

if (mode == NRF24_MODE_TX)
{
    // CONFIG: EN_CRC=1, CRCO=1, PWR_UP=1, PRIM_RX=0
    NRF24_WriteReg(NRF24_REG_CONFIG, 0x0E);
    Delay_ms(2);
    NRF24_CE_L();          // CE<10μs → PTX 发射
}
else // RX mode
{
    // CONFIG: EN_CRC=1, CRCO=1, PWR_UP=1, PRIM_RX=1
    NRF24_WriteReg(NRF24_REG_CONFIG, 0x0F);
    Delay_ms(2);
    NRF24_CE_H();          // CE=高 → 开始监听
}
return 1;
}

```

CONFIG 寄存器 0x0F 的 bit 含义:

bit 0 (PRIM_RX)	1 = 接收模式
bit 1 (PWR_UP)	1 = 上电
bit 2 (CRCO)	1 = 2 字节 CRC
bit 3 (EN_CRC)	1 = 使能 CRC
bit 4~7	保留/MASK 位, 此处为 0

2.2.5 遥控数据包协议

自定义 12 字节固定载荷协议:

偏移	字段	类型	说明
0~1	throttle	uint16_t	油门, 1000~2000μs 对应 0~100%
2~3	roll	int16_t	横滚, -500~500→- 100~100%
4~5	pitch	int16_t	俯仰
6~7	yaw	int16_t	偏航
8	sw1	uint8_t	开关 1 (用作武装开关)
9	sw2	uint8_t	开关 2 (备用通道)
10~11	seq + checksum	uint8_t×2	递增序号 + 前 11 字节和校验

校验实现:

```
static uint8_t RC_Checksum8(const uint8_t *buf, uint8_t len)
{
    uint8_t sum = 0;
    for (uint8_t i = 0; i < len; i++) sum += buf[i];
    return sum;
}

// 发送端打包: 清零 checksum → 计算校验和
uint8_t RC_PacketPack(RC_CtrlPacket_t *pkt)
{
    pkt->checksum = 0;
    pkt->checksum = RC_Checksum8((uint8_t *)pkt, NRF24_PAYLOAD_SIZE - 1);
    return 1;
}

// 接收端校验: 重新求和 → 对比 checksum
uint8_t RC_PacketCheck(const RC_CtrlPacket_t *pkt)
{
    uint8_t sum = RC_Checksum8((const uint8_t *)pkt, NRF24_PAYLOAD_SIZE - 1);
    return (sum == pkt->checksum) ? 1 : 0;
}
```

数据到物理量的映射 (main.c:NRF24_ReceiveRocketData) :

```
// 油门: 1000~2000us → 0~100%
pRocket->throttle = ((float)(rc_pkt.throttle - 1000) / 1000.0f) * 100.0f;

// 横滚/俯仰/偏航: -500~500 → -100~100%
pRocket->roll_input  = ((float)rc_pkt.roll / 500.0f) * 100.0f;
pRocket->pitch_input = ((float)rc_pkt.pitch / 500.0f) * 100.0f;
pRocket->yaw_input   = ((float)rc_pkt.yaw / 500.0f) * 100.0f;

// 武装标志
pRocket->armed = (rc_pkt.sw1 != 0) ? 1 : 0;
```

2.2.6 中断驱动接收

NRF24L01 的 IRQ 引脚 (PB0) 在收到新数据包时将输出低电平。配置为 EXTI0 下降沿触发中断, ISR 给出二值信号量唤醒 Rocket_Control_Task。

```
// EXTI0/NVIC 配置
EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;    // 下降沿
NVIC_InitStructure.NVIC_IRQChannel = EXTI0_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 12; // 安全优先级

// ISR
void EXTI0_IRQHandler(void)
{
    if(EXTI_GetITStatus(EXTI_Line0) != RESET)
    {
        EXTI_ClearITPendingBit(EXTI_Line0);
        BaseType_t xWoken = pdFALSE;
        xSemaphoreGiveFromISR(xNrf24Semaphore, &xWoken);
        portYIELD_FROM_ISR(xWoken);
    }
}

// 任务
void Rocket_Control_Task(void *pvParameters)
{
    while(1)
    {
        // 等待中断信号量, 200ms 超时 = 通信丢失
        if(xSemaphoreTake(xNrf24Semaphore, pdMS_TO_TICKS(200)) == pdTRUE)
        {
            while(NRF24_ReceiveRocketData(&temp_rocket))
            {
                // 耗尽 RX FIFO, 每次取最新的滤波
                temp_rocket.throttle = LowPassFilter(...);
                // ...
            }
        }
        else
        {
            // ...
        }
    }
}
```

```
{
    // 200ms 无数据 → 通信丢失: disarm, 清零
    temp_rocket.throttle = 0.0f;
    temp_rocket.armed = 0;
}
xQueueSend(rocket_queue, &temp_rocket, 0);
}
```

3. 控制算法模块设计

3.1 PID 控制器

涉及文件：HardWare/pid.c / .h （共 110 行 + 33 行头文件）

3.1.1 原理分析

PID（比例-积分-微分）控制器是工业控制中最广泛使用的反馈控制算法。其输出由三项加权求和得到：

$$u(t) = K_p \times e(t) + K_i \times \int e(\tau) d\tau + K_d \times de(t)/dt$$

其中 $e(t) = \text{target} - \text{measure}$ 为误差信号。

比例项（P）：与当前误差成正比，误差越大纠正力越强，但存在稳态误差。

积分项（I）：累积历史误差，消除稳态误差，但过大会导致 overshoot 和 windup。

微分项（D）：预测误差变化趋势，增加阻尼，抑制 overshoot，但放大噪声。

本项目采用位置式 PID（非增量式），即直接计算绝对值输出。位置式相比增量式的优点是输出直接对应物理量（力矩），便于限幅。

3.1.2 代码实现

```
float PID_Output(PID_t *pid, float target, float measure, float dt)
{
    float error = target - measure;

    pid->integral += dt * error;                // 积分累加
    pid->integral = PID_limit(pid->integral, pid->integral_limit); // 积分限幅

    float derivative = (error - pid->last_error) / dt; // 微分（后向差分）

    float output = pid->kp * error
                  + pid->ki * pid->integral
                  + pid->kd * derivative;          // 加权求和
    output = PID_limit(output, pid->output_limit); // 输出限幅

    pid->last_error = error;
    return output;
}
```

注意积分项必须使用 += 累加（而非 = 赋值），否则积分项恒等于 $K_i \times dt \times e(t)$ ，本质上退化为比例项的缩放，完全失去消除稳态误差的能力。此 Bug 曾在项目 pd.c:27 中发现并修复。

3.1.3 参数含义与限幅

PID 结构体定义：

```
typedef struct
{
    float kp;           // 比例增益
    float ki;           // 积分增益
    float kd;           // 微分增益
    float integral;      // 积分累积值
    float last_error;    // 上一次误差（用于微分）
    float integral_limit; // 积分限幅（防 windup）
    float output_limit;  // 输出限幅
} PID_t;
```

积分限幅（integral_limit）和输出限幅（output_limit）是防 windup 和保安全的双重保护：

先将积分累积限制在 $\pm \text{integral_limit}$ 内防止积分过度膨胀，再对最终输出使用

$\pm \text{output_limit}$ 防止电机指令越界。

PID 实例	Kp	Ki	Kd
pid_roll_angle	4.0	0.0	0.8
pid_pitch_angle	4.0	0.0	0.8
pid_roll_rate	0.8	0.0	0.08
pid_pitch_rate	0.8	0.0	0.08
pid_yaw_rate	1.2	0.0	0.0

3.2 串级飞控 (fly_ctrl)

涉及文件：HardWare/fly_ctrl.c / .h （共 111 行 + 35 行头文件）

3.2.1 串级控制原理

串级 PID (Cascade PID) 将两个 PID 控制器串联：外环 PID 的输出作为内环 PID 的期望值（设定点），形成内外嵌套结构。

外环（角度环）目标：让飞行器的实际角度跟随遥控器给定的目标角度（如 Roll=30°）。

内环（角速度环）目标：让飞行器的实际角速度跟随外环给出的目标角速度。

优势：内环能快速响应和抑制扰动（如阵风），外环负责稳定角度。扰动在影响角度之前已被内环衰减，控制品质优于单环 PID。

3.2.2 外环（角度环）

```
// 外环：角度误差 → 目标角速度
target_gx = PID_Output(&pid_roll_angle,
                        fly_target.target_roll,    // 期望角度（来自遥控器）
                        angle.Roll,               // 实际角度（来自互补滤波）
                        dt);

target_gy = PID_Output(&pid_pitch_angle,
                        fly_target.target_pitch,
                        angle.Pitch,
                        dt);
```

角度环 $K_p=4.0$ ：每 1° 角度误差产生 4°/s 的目标角速度。例如飞机右倾 5° → 外环输出 20°/s 左滚的目标角速度给内环。

3.2.3 内环（角速度环）

```
// 内环：角速度误差 → 电机修正力矩
roll_out = PID_Output(&pid_roll_rate,
                      target_gx,                // 目标角速度（来自外环输出）
                      gyro.GYRO_X,              // 实际角速度（来自陀螺仪）
                      dt);

pitch_out = PID_Output(&pid_pitch_rate,
                       target_gy,
                       gyro.GYRO_Y,
                       dt);

yaw_out = PID_Output(&pid_yaw_rate,
                     fly_target.target_yaw_rate, // Yaw 仅速率控制
```



```
gyro.GYRO_Z,  
dt);
```

偏航轴 (Yaw) 特殊处理：由于无磁力计，无法获取绝对偏航角度，因此没有角度外环，仅使用内环做角速率控制。飞手打偏航杆时控制的是偏航角速度 ($^{\circ}/s$)，而非偏航角度。

3.2.4 电机混控矩阵

X 型四轴布局（机身朝向在两电机对角线之间），4 个电机的转速由油门、俯仰、横滚、偏航四路控制量组合得到：

```
motor_out.m1 = throttle - pitch_out - roll_out - yaw_out;  
motor_out.m2 = throttle - pitch_out + roll_out + yaw_out;  
motor_out.m3 = throttle + pitch_out + roll_out - yaw_out;  
motor_out.m4 = throttle + pitch_out - roll_out + yaw_out;
```

物理含义：

电机	油门	俯仰(前倾+)	横滚(右倾+)	偏航(右转+)
M1 (FR)	base	-pitch	-roll	-yaw
M2 (FL)	base	-pitch	+roll	+yaw
M3 (RR)	base	+pitch	+roll	-yaw
M4 (RL)	base	+pitch	-roll	+yaw

限幅：每个电机输出截断到 $[1000, 2000]\mu s$ ，对应标准航模电调的有效输入范围。

4. 底层驱动模块设计

4.1 软件 I2C

涉及文件：System/i2c.c / .h （共 363 行 + 53 行头文件）

4.1.1 STM32F1 硬件 I2C 缺陷

STM32F103 的硬件 I2C 外设存在设计缺陷（ST 官方勘误表 ES055 中确认）：当从机在传输过程中执行时钟拉伸时，I2C 外设的状态机可能进入非法状态，导致 SCL 线被持续拉低、总线锁死。通常需要通过复位 I2C 外设甚至重新初始化 GPIO 来恢复，严重影响实时性。

因此项目未使用 STM32 的硬件 I2C，而是用 GPIO 比特率方式完全模拟 I2C 时序。

4.1.2 GPIO 比特率模拟原理

I2C 总线由两根线组成：SCL（时钟）和 SDA（数据），通过开漏输出 + 上拉电阻实现电平驱动。总线空闲时 SCL 和 SDA 均为高电平。使用 GPIO 的 BSRR（置高）和 BRR（置低）寄存器直接控制引脚状态，实现时序。

```
// GPIO 电平控制宏
#define SDA_High()    (I2C_PORT->BSRR = I2C_SDA_PIN_MASK)
#define SDA_Low()     (I2C_PORT->BRR  = I2C_SDA_PIN_MASK)
#define SCL_High()    (I2C_PORT->BSRR = I2C_SCL_PIN_MASK)
#define SCL_Low()     (I2C_PORT->BRR  = I2C_SCL_PIN_MASK)
#define SDA_Read()    ((I2C_PORT->IDR & I2C_SDA_PIN_MASK) ? 1 : 0)
#define SCL_Read()    ((I2C_PORT->IDR & I2C_SCL_PIN_MASK) ? 1 : 0)
```

SCL 频率通过 Delay_us(5) 控制：每 bit 2 次 SCL 翻转（高+低） $\times 5\mu\text{s} = 10\mu\text{s} \rightarrow 100\text{kHz}$ 。

完全满足标准 I2C 模式（100kHz）的时序要求。

4.1.3 起始/停止条件

起始条件（SCL 高电平时 SDA 从高→低），停止条件（SCL 高电平时 SDA 从低→高）：

```
void Soft_I2C_Start(void)
{
    Soft_SDA_OutPut();
    SDA_High();
    SCL_High();
    Delay_us(5);    // 建立时间
    SDA_Low();      // SCL 高时 SDA 下降沿 = Start
```

```

    Delay_us(5);
    SCL_Low();    // 拉低 SCL 准备传输数据
}

void Soft_I2C_Stop(void)
{
    Soft_SDA_OutPut();
    SDA_Low();
    Delay_us(1);
    SCL_High();
    Delay_us(5);
    SDA_High();    // SCL 高时 SDA 上升沿 = Stop
    Delay_us(5);
}

```

4.1.4 字节读写与时序

发送一字节 (MSB first) + 等待应答:

```

void Soft_I2C_SendByte(uint8_t data)
{
    Soft_SDA_OutPut();
    for (uint8_t i = 8; i > 0; i--)
    {
        if (data & 0x80) SDA_High(); else SDA_Low(); // 设置 SDA
        data <<= 1;
        Delay_us(5);
        SCL_High();    // SCL 高 → 从机采样 SDA
        Delay_us(5);
        SCL_Low();    // SCL 低 → 准备下一个 bit
    }
    SDA_High();    // 释放 SDA, 让从机控制应答
}

uint8_t Soft_I2C_WaitAck(void)
{
    uint16_t timeout = 0;
    SDA_High();
    Soft_SDA_InPut();    // SDA 切换为输入
    Delay_us(5);
    SCL_High();    // 第 9 个时钟高
    Delay_us(5);

    while(SDA_Read())    // 等待从机拉低 SDA 确认应答
    {
        if(++timeout > 200)    // 200μs 超时保护
        {
            SCL_Low();
            Soft_SDA_OutPut();
            return 1;    // 无应答
        }
    }
}

```

```

    }
    SCL_Low();
    Soft_SDA_OutPut();
    return 0;          // 收到应答
}

```

读取一字节:

```

uint8_t Soft_I2C_ReadByte(void)
{
    uint8_t data = 0;
    Soft_SDA_InPut();    // SDA 切换为输入 (由从机驱动)
    for (int i = 0; i < 8; i++)
    {
        data <<= 1;
        SCL_High();
        Delay_us(5);
        if(SDA_Read()) data |= 0x01; // 上升沿采样
        SCL_Low();
        Delay_us(5);
    }
    Soft_SDA_OutPut();
    return data;
}

```

4.1.5 多字节传输

多字节读取时需要发送重新起始条件 (Repeated Start) , 在读取最后一个字节后发送 NACK (非应答) 通知从机停止发送。

```

uint8_t Soft_I2C_Read(uint8_t slave_addr, uint8_t reg,
                      uint8_t *buf, uint8_t len)
{
    if ((buf == 0) && (len != 0)) return 0;

    Soft_I2C_Start();
    Soft_I2C_SendByte(slave_addr << 1);    // 写方向
    if(Soft_I2C_WaitAck()) { Soft_I2C_Stop(); return 0; }

    Soft_I2C_SendByte(reg);                // 寄存器地址
    if(Soft_I2C_WaitAck()) { Soft_I2C_Stop(); return 0; }

    Soft_I2C_Start();                      // Repeated Start
    Soft_I2C_SendByte((slave_addr << 1) | 0x01); // 读方向
    if(Soft_I2C_WaitAck()) { Soft_I2C_Stop(); return 0; }

    for(uint8_t i = 0; i < len; i++)
    {
        buf[i] = Soft_I2C_ReadByte();
        if (i == len-1) Soft_I2C_NoAck();    // 最后字节 NACK
        else Soft_I2C_Ack();
    }
    Soft_I2C_Stop();
}

```

```

    return 1;
}

```

Soft_I2C_SendByte_Point 用于 MPU6050 写寄存器操作，流程为：Start → 发送地址+W → 等待应答 → 发送寄存器地址 → 等待应答 → 发送数据 → 等待应答 → Stop。逐级检查应答信号，任何一阶失败即终止并返回错误。

4.2 PWM 输出

涉及文件：System/pwm.c / .h （共 92 行 + 21 行头文件）

PWM 输出使用 TIM2 和 TIM4 定时器，每路 PWM 独立控制电调/电机转速。

电机	定时器/通道	GPIO
M1	TIM2_CH3	PA2
M2	TIM2_CH4	PA3
M3	TIM4_CH3	PB8
M4	TIM4_CH4	PB9

定时器配置参数：

```

// 预分频 72-1: 72MHz / 72 = 1MHz (1μs 计数一次)
TIM_TimeBaseStructure.TIM_Prescaler = 72 - 1;

// ARR = 20000-1: PWM 周期 = 20000 × 1μs = 20ms → 50Hz
TIM_TimeBaseStructure.TIM_Period = 20000 - 1;

```

// PWM 比较值范围 1000~2000 → 1~2ms 脉宽（标准航模电调协议）

ARR = 20000-1 对应 50Hz PWM 频率（标准航模电调协议），CCR 值 1000~2000 映射到 1~2ms 脉宽。过初始版本中错误地设置为 50-1（20kHz），导致 1000~2000 的 CCR 值远超 ARR=49，电机实际输出始终为最大占空比。此 Bug 在 V2.0 中修复。

4.3 UART 调试与命令行

涉及文件：System/usart.c / .h, System/cli.c / .h

使用 USART1 (PA9 TX, PA10 RX) , 波特率 115200bps。通过重定向标准库的 fputc 实现 printf 直接输出到串口。

Monitor_Task 每 200ms 输出以下调试信息：遥控输入（油门/横滚/俯仰/偏航/武装状态）、姿态数据（Pitch/Roll/Yaw 角度）、角速度（GX/GY/GZ）、电机输出（M1~M4）。

命令行接口 (CLI) 功能：

V2.0 新增 UART 命令行接口，支持以下交互式命令：

```
STM32> help
Available commands:
  help          - Show help
  pid list       - List all PID gains
  pid get <name> - Get specific PID gains
  pid set <name> <kp> <ki> <kd> - Set PID gains
  pid reset <name> - Reset PID integral
  status        - Show motor outputs
PID names: roll_angle, pitch_angle, roll_rate, pitch_rate, yaw_rate
```

实现架构：USART1 使能 RXNE 中断，ISR 将接收字符存入 256 字节环形缓冲区。CLI_Task（优先级 1）每 10ms 轮询读取缓冲区，逐字符回显并缓存到命令行缓冲区。遇到回车符（\r\n）时解析命令并执行。

PID 调参命令通过 fly_ctrl.c 导出的访问器函数（如 Fly_GetRollAnglePID()）直接修改 PID 结构体的 kp/ki/kd 参数，可实现飞行中实时调参，无需重新编译。

4.4 SD 卡黑盒记录仪

涉及文件：HardWare/logger.c / .h（SPI2 + 原始扇区写入）

V2.0 新增 SD 卡飞行数据记录功能，使用 SPI2 接口（PB13-SCK, PB14-MISO, PB15-MOSI, PB12-CS）与 SD 卡通信，以原始扇区写入方式记录飞行数据。

记录的数据结构（每 48 字节）：

偏移	字段	说明
0~3	magic (uint32_t)	标识: 0x4C4F4745

		("LOGE")
4~7	seq (uint32_t)	递增序列号
8~11	timestamp_ms (uint32_t)	系统运行时间 (ms)
12~15	roll (float)	横滚角 (°)
16~19	pitch (float)	俯仰角 (°)
20~23	yaw (float)	偏航角 (°)
24~27	gyro_x (float)	X 轴角速度 (°/s)
28~31	gyro_y (float)	Y 轴角速度 (°/s)
32~35	gyro_z (float)	Z 轴角速度 (°/s)
36~39	m1 (float)	电机 1 输出 (μs)
40~43	m2 (float)	电机 2 输出 (μs)
44~47	m3 (float)	电机 3 输出 (μs)
48~51	m4 (float)	电机 4 输出 (μs)

SD 卡初始化流程 (SPI 模式) :

1. SPI2 初始化 (CPOL=0, CPHA=0, 分频=256 → 281kHz)
2. 74+ 个空闲时钟 (CS=高)
3. CMD0 (GO_IDLE_STATE) → R1=0x01
4. CMD8 (SEND_IF_COND) → 验证电压范围 + 0xAA 模式
5. CMD55 + ACMD41 (SD_SEND_OP_COND) → 等待 R1=0x00
6. CMD16 (SET_BLOCKLEN) → 设为 512 字节块
7. 扫描扇区查找最后一个有效扇区 (magic 匹配)

写入策略:

每 5ms (200Hz) 从 Fly_Control_Task 收集一次数据, 通过 FreeRTOS 队列 (容量 10) 发送给 LOG_Task。

LOG_Task 将数据暂存在 512 字节扇区缓冲区中, 每满 10 条记录 ($512/48=10.67$, 实际取 10 条) 写入一个扇区。

写入使用 CMD24 (WRITE_BLOCK) , 等待 card 返回 0x05 (数据接受) + 0x00 (空闲)。

日志为循环覆盖模式, 从 boot 开始扫描最后一个有效扇区后继续写入。

5. 优化过程与 Bug 修复

5.1 阶段一：功能性 Bug 修复

在代码审查中发现并修复了以下 6 个严重问题：

问题	文件位置	修复方式
PID 积分累加使用 = 而非 +=	pid.c:27	将赋值改为累加，恢复积分作用
限幅语句全部写为 m1	fly_ctrl.c:80-83	改为各自的 m1/m2/m3/m4
PWM_Init() 在 main 中未调用	main.c	初始化流程中添加 PWM_Init()
传感器队列 xQueueSendFromISR 被注释	mpu6050.c	取消注释并补全实现
MPU6050_INT_SetQueueHandle 未调用	main.c	添加调用，并启用中断信号量方案
NRF24 CE 引脚(PA2)与 TIM2_CH3 冲突	nrf24l01.h	CE 从 PA2 改为 PA1

5.2 阶段二：中断驱动重构

两个关键的中断优化：

- NRF24L01 轮询改中断：

维度	轮询方案	中断方案
触发方式	固定 20ms 周期读取	IRQ(PB0) 下降沿触发

	STATUS	
SPI 流量	每次任务切换读 STATUS (即使无数据)	仅数据到达时读载荷
CPU 占用	每 20ms 唤醒一次	无数据时完全阻塞

- MPU6050 ISR 轻量化:

操作	优化前 (ISR 内)	优化后 (任务内)
I2C 读取 14 字节	~1.26ms, 阻塞所有任务和中断	Sensor_Read_Task 中执行
atan2f+sqrtf 浮点	~0.3ms, 无硬件 FPU	Sensor_Read_Task 中执行

5.3 阶段三：任务职责重划

任务	原始职责	优化后职责
RocketCtrl	20ms 轮询 NRF24 + 低通滤波	等待信号量 → 耗尽 FIFO + 滤波
FlightCtrl	含 MPU6050_AngleUpdate (姿态更新)	仅 PID 控制 (姿态由 SensorRead 更新)
SensorRead	每秒 peek 一次队列 (无实际作用)	5ms 周期 I2C 读取 + Mahony 滤波 + 发队列
MPU6050 ISR	I2C 读取 + atan2f + 发队列	仅清 EXTI + 给信号量

5.4 阶段四：V2.0 功能增强

在 V2.0 版本中进行了以下扩展功能开发：

功能	实现方式	涉及文件
Mahony 滤波器	四元数 PI 反馈滤波替代固定权重互补滤波	HardWare/Mahony.c, mpu6050.c
PWM ARR 修复	TIM_Period = 50-1 → 20000-1 (50Hz 标准电调协议)	System/pwm.c
UART 命令行 CLI	USART1 RXNE 中断 + 环形缓冲区 + 任务解析	System/cli.c, usart.c
SD 卡黑盒日志	SPI2 接口 + 原始扇区读写 + 队列触发	HardWare/logger.c
PID 运行时调参	通过 CLI 命令在飞行中修改 Kp/Ki/Kd	fly_ctrl.c (PID 访问器)

5.5 各阶段收益汇总

阶段	改进内容	收益
阶段一	修复 6 个功能性 Bug	代码可以正常运行, 电机响应可控
阶段二	NRF24/M PU6050 中断驱动	CPU 利用率降低 ~60%, ISR 从 ~1.5ms 缩短至 ~2μs
阶段三	任务职责重划	I2C/浮点运算从 ISR 移至任务, 系统稳定性大幅提升
阶段四	Mahony + CLI + 日志 + PWM 修复	姿态精度提升、可调参调试、飞行数据可追溯、电机按预期工作

6. 痛点设计与决策权衡

6.1 中断优先级与 FreeRTOS 安全边界

FreeRTOS 通过 ARM Cortex-M3 的 BASEPRI 寄存器保护临界区。设置 BASEPRI=191 后，优先级数值 >= 191 的中断被屏蔽，数值 < 191 的中断可以抢占内核。

中断优先级设计规则（需满足 NVIC_PriorityGroup_4）：

- ① 调用任何 FreeRTOS API 的中断优先级必须 >= configMAX_SYSCALL_INTERRUPT_PRIORITY（原始值 191，对应库优先级 11）
- ② 中断优先级数值越小，实际优先级越高
- ③ EXTI4_IRQn 和 EXTI0_IRQn 配置为库优先级 12，低于安全边界，可调用 API
- ④ 项目启动时需调用 NVIC_PriorityGroupConfig(NVIC_PriorityGroup_4)，否则优先级分组不确定

项目中各中断优先级分配：

中断	库优先级	对应的 FreeRTOS API
EXTI0 (NRF24 IRQ)	12	xSemaphoreGiveFromISR
EXTI4 (MPU6050)	12	xSemaphoreGiveFromISR

6.2 引脚冲突与资源规划

STM32F103C8 的 LQFP48 封装可用 GPIO 较少。项目初期 NRF24_CE 和 TIM2_CH3(PWM1) 同时占用 PA2，造成功能冲突。修复方案：将 NRF24_CE 从 PA2 改至空闲的 PA1。

GPIO	功能	复用
PA1	NRF24_CE	—

PA2	TIM2_CH3 (PWM M1)	—
PA3	TIM2_CH4 (PWM M2)	—
PA4	NRF24_CSN	SPI1_NSS(软件)
PA5	SPI1_SCK	—
PA6	SPI1_MISO	—
PA7	SPI1_MOSI	—
PB8	TIM4_CH3 (PWM M3)	—
PB9	TIM4_CH4 (PWM M4)	—

6.3 姿态估计算法选型

V2.0 从互补滤波升级为 Mahony 滤波器，以下是选型决策分析：

维度	互补滤波 (V1.0)	Mahony 滤波器 (V2.0)
核心思路	固定权重高通+低通	PI 反馈修正 + 四元数互补
计算量/周期	~30 μ s (F103 无 FPU)	~50 μ s (F103 无 FPU)
抗加速度干扰	差 (固定权重 0.98/0.02)	好 (误差反馈自动调节)
横滚/俯仰精度	悬停可接受	机动时显著更优
Yaw 处理	纯积分 (必然漂移)	纯积分 (无磁力计, 同互补)